

Classification-based Technique to detect Social Engineering Attacks using Continuous Data Stream Machine Learning Algorithms

Admilson de Ribamar Lima Ribeiro¹ and João Marco Cardoso da Silva²

¹ Computing Department, Federal University of Sergipe & INESC TEC, São Cristóvão, Brazil
email: admilson@dcomp.ufs.br

² Department of Informatics, University of Minho & INESC TEC, Braga, Portugal
email: joaomarco@di.uminho.pt

Abstract — Social engineering attacks are designed to manipulate individuals and organizations into revealing valuable and confidential information for the benefit of cybercriminals. Machine learning approaches based have been developed to detect the malicious activity of these attacks. However, they are limited by the use of static data found in datasets. This limitation affects the accuracy of the algorithm because the training dataset quickly becomes outdated, causing errors in the estimator's prediction. Furthermore, model training is done offline due to the high computational cost of this activity. Therefore, in this paper, we analyze the use of machine learning algorithms over continuous data streams. This type of algorithm is suitable for use on the Web due to the flow treatment given to network traffic and allows online training of the estimator to detect new attacks. Our approach eliminates the drawbacks of traditional machine learning techniques that use batch algorithms. To characterize the anomalies, we analyze the pretexting attack that can occur on the Web. We adapt the K-Nearest Neighbors, Bernoulli Naive Bayes, Logistic Regression and Hoeffding Adaptive Tree Classifier algorithms to handle continuous data streams to verify which the one has the best effectiveness and efficiency in online learning. The experimental results show that these models can be used efficiently and effectively in online training.

Keywords - Detection, SE attacks, Machine Learning, Data Stream, Online training.

I. INTRODUCTION

Since 2020, with the start of the COVID-19 pandemic, the job market has undergone a major transformation. One of the changes is that most corporate employees now work remotely from home, and this fact has led to new breaches in computer security. In general, people trust other people more than they trust computers or technology. This makes them the weakest link in the security chain. Malicious activities carried out through human interactions psychologically influence a person to disclose confidential information or break security procedures. These malicious activities characterize social engineering attacks [1].

As organizations have worked hard to protect their data in times of rapid change, cyber threats have adapted. While the nature of the threats has remained virtually the same, the scale

and scope of the attacks have increased significantly, and the vast majority of these threats are social engineering attacks. As a result, users need protection from both social engineering attacks and the possibility that their devices are supporting attacks on organizations. So we need techniques to detect these attacks.

Several machine learning techniques have been proposed and described in the literature for detecting social engineering attacks. These techniques mainly generate the learning model from tabular data. Learning from tabular data is part of so-called batch learning [2], which basically means that all data is available to the learning algorithm at once. However, this approach has some disadvantages.

First, a computer may crash if a given routine returns a data set whose size exceeds the available amount of RAM. Another problem with the batch learning model is that it cannot learn efficiently from new data. In fact, when new data becomes available, the model must learn from scratch with a new data set composed of the old data and the new data. A final drawback concerns the way features are extracted. Every time you want to train a model, you first have to extract features. The problem is that some features may not be available at the time they are needed.

Our solution to overcome these drawbacks is to use online learning [2] to detect social engineering attacks. The goal of online learning is to fit a model to a stream of data. In other words, the data is not fully available, but the observations are provided one by one. Therefore, in this paper we present a technique to detect social engineering attacks using machine learning algorithms on continuous data streams. Since the samples of social engineering attacks are in textual data, the features necessary for training the algorithm were extracted using Natural Language Processing (NLP) techniques. We analyzed the K-Nearest Neighbors (KNN), Bernoulli Naive Bayes (BernoulliNB), Logistic Regression (LR), and Hoeffding Adaptive Tree Classifier (HATC) algorithms to verify the one that has the best effectiveness and efficiency in online learning.

In this context, we have divided the paper into five further sections. In Section II, we present related work and highlight our contribution. In Section III, we describe the main social engineering attacks that can occur in an online environment. In Section IV, we present the main techniques for detecting

social engineering attacks. In Section V, we present the experimental methodology used to conduct the experiment, describe the experimental evaluation, and discuss the results obtained (along with the validity threats) of such an experiment. Finally, in Section VI, we point out future research and conclude the paper.

II. RELATED WORK

There are several papers in the literature that combine the machine learning approach with natural language processing techniques to develop methods for detecting social engineering attacks.

In [3], the authors present a method for detecting malicious statements contained in emails using natural language processing techniques. The method analyzes the statements and discriminates the malicious ones that imply a phishing attack. The approach relies on analyzing the text, rather than the metadata that may be associated with emails. As a result, this approach is effective for detecting phishing emails composed of plain text. The proposed algorithm is evaluated using a dataset of phishing emails that is used as a reference set.

In [4], the authors use machine learning techniques to detect grooming attacks. The study shows the use of a dataset containing grooming conversations and then extracting features from these conversations using natural language processing techniques. The classification process then determines whether the conversation is a grooming attack or not.

In [5], the authors use natural language processing techniques and artificial neural networks to detect social engineering attacks. They describe a method where offline or online text documents are processed using natural language processing tools and artificial neural networks are used to discriminate whether the text is a social engineering attack or not. The proposed method showed high accuracy results during the evaluation, where a real and a semi-synthetic dataset were used for model training. In addition, several other classification models were used to compare the two datasets. This study extends the practical work presented in [6] and [7], where the authors also use natural language processing and machine learning to successfully detect social engineering attacks.

In [8], the author proposes a social engineering detection model based on deep neural networks. The model is capable of detecting deception and phishing attempts by analyzing textual content. In the first stage, the chat history is processed and analyzed using natural language processing techniques, and the context semantics are captured and extracted using a Bidirectional Long and Short Term model (bi-LSTM). The integration of user characteristics and chat content as features for classification is done by ResNet [9].

However, these studies treat the problem of detecting social engineering attacks as a classification problem, where there are two classes: "normal" and "non-normal". The goal is to determine which of the two classes best fits the given set of examples. Due to the need for a set of examples to estimate the output, the classification task is categorized as a supervised approach [10]. The traditional supervised approach

uses algorithms that are limited by the use of static data typically found in datasets. These algorithms use training data sets that are available in memory. In some environments, such as IoT applications, TCP/IP traffic, e-commerce, and websites that generate high-speed data streams, it is not feasible to store all the data in memory and perform multiple passes over the training data. In addition, the learning model can change over time due to the generation of data by non-stationary distributions, such as the appearance of new malicious activity on the computer network. This change affects the accuracy of the algorithm because the training set quickly becomes outdated, resulting in errors in the estimator's prediction. In addition, model training is done offline due to the high computational cost of this activity.

Also, consider that social engineering attacks on the Web usually occur through applications that are fed not by static data, but by data streams. Therefore, in this case, the best way to analyze the collected information is not through datasets, but based on transient data streams. Thus, there is a need for a new approach that takes into account continuous data streams in social engineering attacks. Data stream machine learning algorithms are suitable for use on the Web due to the flow treatment given to network traffic, and also allow online training of the estimator to detect new social engineering attacks.

III. MAIN SOCIAL ENGINEERING ATTACKS

Different descriptions of social engineering attacks that appear in the literature give different ideas of what a social engineering attack entails. Some of these descriptions focus specifically on obtaining information from an organization. Other descriptions describe a social engineering attack as manipulating and persuading people to obtain information or to take some action. In addition, some of the descriptions focus on gaining access to computer systems and networks. The only element common to all these descriptions is that a person is exploited to obtain some unauthorized information or to perform some action. Thus, we can state that social engineering attacks are based on the use of social interaction as a means to persuade an individual or an organization to comply with a specific request from an attacker, when the social interaction, persuasion, or request involves an entity related to the computer [11].

We can classify social engineering attacks in several ways: by the operator performing the attack, the method of deception, the type of connection, the communication channel, or the vector that must be triggered to initiate the attack. In the case of operator-based classification, the perpetrator of the attack can be a human or a digital computer. The main human-based attacks are [11]: face-to-face interaction, shoulder surfing, quid pro quo, dumpster diving, diversion theft, tailgating, among others. The main computer-based attacks that can be addressed by machine learning are [11]: phishing, pretexting, ransomware, profile cloning, and online grooming.

Pretexting attacks, the subject of this study, consist of inventing false and convincing scenarios to steal the victim's personal information. They are based on pretexts that make

the victim believe and trust the attacker [11]. The attack is carried out through phone calls, emails, or physical media.

IV. SOCIAL ENGINEERING ATTACKS DETECTION TECHNIQUES

Several techniques have been proposed in the literature to combat social engineering attacks. A list of defensive measures against social engineering attacks includes: promoting security education and training, raising social awareness of social engineering attacks, providing the necessary tools to detect and prevent these attacks, learning how to keep confidential information secure, reporting any suspicious activity to the security service, organizing security policies for new employees and disclosing the risks of attacks to all employees, forwarding awareness emails and known fraudulent emails [12].

In this context, we present a machine learning-based method for detecting social engineering attacks. These methods employ techniques to detect anomalies in a network environment. These techniques can be model-based, based on statistics and mathematics, based on unsupervised machine learning algorithms and based on supervised machine learning algorithms [13].

The algorithms used in machine learning take into account various network features and traffic characteristics to detect the presence of anomalies. In fact, any system built to detect an anomaly in the network captures the traffic on it and extracts some kind of information from it. The machine learning approach attempts to capture the normal and abnormal traffic patterns on the network, without needing to know the pattern itself.

V. EXPERIMENTAL METHODOLOGY

To carry out the experimental evaluation of our technique, we followed the methodology presented in [14], where the experimental process can be divided into the following main activities: scope determination; planning; operation; analysis and interpretation; and presentation and packaging.

A. Scope Determination

The main objective of the experiment is to analyze ways to effectively and efficiently solve the problem of detecting social engineering attacks in the online environment. A secondary goal is to compare different machine learning algorithms that use data streams as a solution to the problem, analyzing metrics such as accuracy and processing time in online training.

Following the formalization of the GQM model, proposed in [15], both objectives can be unified as: To analyze security methods to identify social engineering attacks in an online environment, with the purpose of comparing them **with** respect to efficacy (accuracy) and efficiency (processing time and memory used) from the point of view of information security researchers and developers in the context of a social engineering attack in an online environment.

B. Experiment Planning

The planning activity is where the foundation for the experiment is laid. This activity has several steps. In the first

step, the context of the experiment is determined in detail. The next step is to determine the variables (both the independent variables (inputs) and the dependent variables (outputs)). The next step is to plan the experiment, which includes choosing an appropriate experimental design. Finally, it is important to consider the validity of the expected results.

Context selection: The experiment was conducted in vitro, using a dataset containing samples of social engineering attacks and also samples of normal traffic. We use two processing modes: batch and online. To simulate an online environment, we adapt the machine learning model to handle data streams. Therefore, the data is not fully available, but the observations are provided one by one. In addition, four different machine learning algorithms were used to detect the anomalies and normal traffic: K-Nearest Neighbors (KNN), Bernoulli Naive Bayes (BernoulliNB), Logistic Regression (LR), and Hoeffding Adaptive Tree Classifier (HATC).

Variable selection: The dependent variables are the accuracy, memory usage, and processing time of each machine learning algorithm. The set of hyperparameters of each algorithm can also be considered as dependent variables because each hyperparameter depends on the data used to train each algorithm.

The independent variables are: the dataset containing samples of social engineering attacks and their parameters, the Python code implemented by the authors, the classification algorithms themselves, and the feature extraction models.

Formulation of hypotheses: The research questions for this experiment are: Can the algorithms (LR, KNN, BernoulliNB or HATC) operating in batch or can the use of those operating over data streams be considered better in terms of efficacy for the general classification of social engineering attacks? Can the algorithms (LR, KNN, BernoulliNB or HATC) operating in batch or the use of those operating over data streams be considered better in terms of efficiency for the general classification of social engineering attacks? Could any of the machine learning algorithms operating on data stream (LR, KNN, BernoulliNB or HATC) be considered better than all others in terms of efficacy in the task of classifying social engineering attacks? What is the best algorithm operating on data stream when considering efficiency (processing time and memory usage)? What is the influence of the feature extraction model on the efficacy of algorithms operating on data stream? What is the influence of the feature extraction model on the efficiency of algorithms operating on data stream?

From these questions, the following hypotheses can be tested. Hypotheses 1 and 2 analyze both processing modes: batch and online. Hypotheses 3 to 6 analyze only the online mode.

Hypothesis 1 (for each of the processing modes)

H0: There is no difference between the efficacy metrics of the algorithms with respect to the processing mode.

H1: Some algorithm has a different efficacy metric with respect to the processing mode.

Hypothesis 2 (for each of the processing modes)

H0: There is no difference between the efficiency metrics of the algorithms with respect to the processing mode.

H1: Some algorithm has a different efficiency metric with respect to the processing mode.

Hypothesis 3

H0: There is no difference between the efficacy metrics of the algorithms that operate on data stream.

H1: Some algorithm has a different metric of efficacy with respect to the others.

Hypothesis 4

H0: There is no difference between the efficiency metrics of algorithms that operate over data streams.

H1: Some of the algorithms have a different efficiency metric compared to the others.

Hypothesis 5 (Behavior of algorithms under the influence of feature extraction)

H0: There is no difference between the efficacy metric of classification algorithms in the presence of different feature extraction models.

H1: There is a difference between the efficacy metric of the classification algorithms.

Hypothesis 6 (Behavior of algorithms under the influence of feature extraction)

H0: There is no difference between the efficiency metric of classification algorithms in the presence of different feature extraction models.

H1: There is a difference between the efficiency metric of the classification algorithms.

Object selection: In the experiment, we used data from publicly available datasets through research papers published in the literature. We emphasize that the databases were used to capture accuracy in order to apply more robust statistical tests to the results obtained. All features were captured by applying feature extraction techniques. The models used were Bag of Words (BoW) [16], Term Frequency - Inverse Document Frequency (TF-IDF) [17] and the spaCy framework (<https://spacy.io/>).

Experimental design: The design of the experiment can be summarized according to the following steps: use of the dataset; preprocessing; application of the feature extraction model; training of the classification model; obtaining the efficacy and efficiency metrics.

Instrumentation: The algorithms used were obtained from libraries of the Python programming language, version 3.11.1. The River library [18] was used to build the machine learning models that operate on data streams, while the Scikit-learn library [19] was used to build the models that operate in batches. The algorithms were run on a MacBook Pro 13" personal computer with 8 GB of RAM, an Intel i7 dual-core processor (3.1 GHz), macOS Monterey 12.7 operating system, and Spyder 5.4.5 software installed (the Python environment in which the experiments were run).

C. Experiment Operation

The operational phase of the experiment consists of four stages: preparation, execution, data collection, and validation.

In the preparation stage, we import data from the dataset. We use 147 samples of social engineering dialogs as described in [20]. Out of these 147 samples, 77 samples are characterized as social engineering attacks. To make up the full dataset, we added 4704 more SMS message entries as

described in [21], none of which are classified as social engineering attacks. Thus, the data contains 4851 entries and 1 feature (i.e., text message body). Social engineering attack messages represent 1.44% of the total dataset. The goal is to predict whether a text message is a social engineering attack or not.

In the execution stage, the tasks were performed according to different treatments: preprocessing, feature extraction, hyperparameter search, and training. We use incremental processing, which consists of processing a data stream one element at a time. In the case of machine learning, this means training a model by feeding it one sample at a time. This is completely opposite to the traditional way of doing machine learning, which is to train a model on an entire batch of data at once.

In the data collection stage, we use the scikit-learn and river libraries, from which it was possible to collect the results and store them in a file for future statistical analysis. In the case of collecting data from the experiments that used the efficiency metrics, the data collection was done by inserting codes to capture the processing time of the classification. The efficacy metrics used were Accuracy and AUC. Both metrics are statistical values that determine how close the ML algorithm is to the ideal; if these values are close to 1, it means that the algorithm has no error and classifies the data perfectly [10].

VI. RESULTS AND DISCUSSIONS

The analysis and interpretation of the results follows and considers the hypotheses described in Section V.B.

A. Analysis and Interpretation

Considering hypotheses 1 and 2, Table I shows the accuracy and processing time for batch and online training of the NB algorithm using the TF-IDF scheme. We can see good results in both cases with an accuracy around 0.98. Despite the differences in processing time, online training is competitive with batch training, especially considering that most of the processing time of batch training was dedicated to downloading the small dataset used. As the amount of data increases, online training demonstrates to be more efficient than batch training.

TABLE I. ACCURACY AND PROCESSING TIME OF THE NB ALGORITHM.

Model	Accuracy	Time (sec)
NB-TFIDF(online)	0.98	4.4
NB-TFIDF(batch)	0.96	1.4

To verify if there are differences in the accuracy metrics of the algorithms that act on the data streams (Hypothesis 3), Table II shows that the LR algorithm obtained an accuracy above 0.92 for the three feature extraction schemes used. We can see that the spaCy scheme (which uses word embeddings) obtained the highest value, with a value of 0.96.

TABLE II. ACCURACY OF THE LR ALGORITHM.

Feature Extraction	Accuracy
SpaCy	0.96
TF-IDF	0.96
BoW	0.92

To verify if there are differences in the AUC metric of the algorithms that act on data streams (Hypothesis 3), Table III shows that for the KNN algorithm, the AUC metric has different values, but still suitable for a binary classification algorithm. We can see that the BoW scheme obtained the highest value, around 0.84.

TABLE III. AUC METRIC OF THE KNN ALGORITHM.

Feature Extraction	AUC
BoW	0.84
TF-IDF	0.79
spaCy	0.69

To check if there are differences in the efficiency metric of the algorithms acting on the data streams considering the feature extraction schemes (Hypothesis 4 and 6). The Fig. 1 shows that the algorithms using the BoW scheme require shorter processing times, while the algorithms using the word embeddings scheme require longer processing times for training and evaluation of the data. For this metric, we can affirm that the LR-BoW model (around 0.5 sec) is the most efficient combination, while the HATC-spaCy model (around 80 sec) is the least efficient combination.

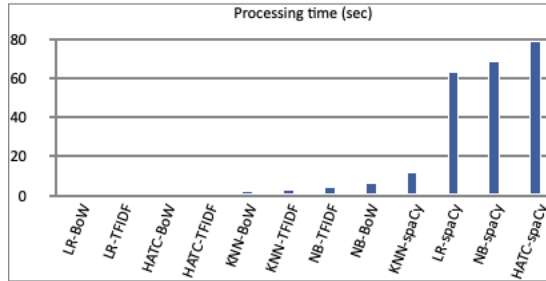


Figure 1. Processing Time of the models

The Fig. 2 shows the influence of the feature extraction scheme on the accuracy metric (Hypothesis 5), we can observe that the KNN-BoW model presented the best result, around 0.98. We can also observe that almost all the models can be trained online for the detection of social engineering attacks, with the exception of the NB-spaCy model because it presented unsatisfactory results, with an accuracy value of 0.03.

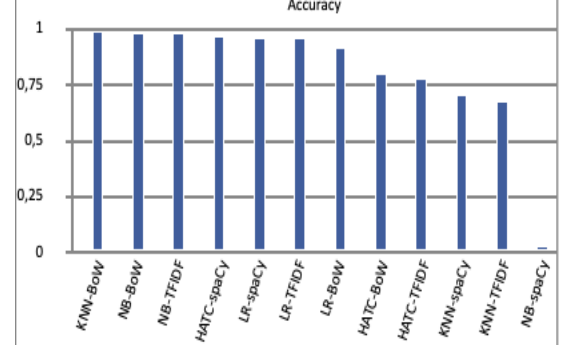


Figure 2. Accuracy of the models

The Fig. 3 shows the influence of the feature extraction scheme on the AUC metric (Hypothesis 5), we can observe that the NB-BoW model presents the best result, with a value of 0.92. Considering this metric, we can say that all models can be trained online for the detection of social engineering attacks, although the NB-spaCy model has a lower result than the other models, around 0.5.

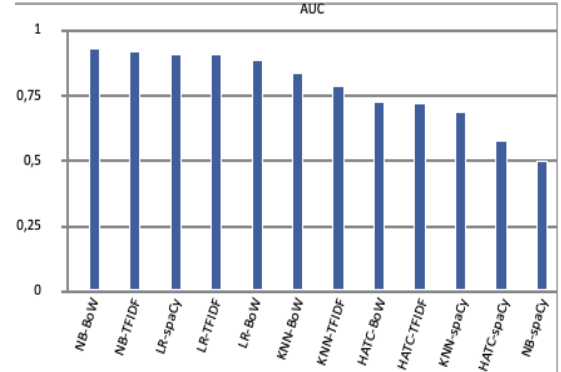


Figure 3. AUC metric of the models

B. Threats to validity

A particular threat to validity is the fact that the data was collected from datasets. In this way, we use proactive data streams, which are those streams that we have control over them. So we can decide at what speed we want to read the data, in what order, etc. This is completely different from reactive data streams where the data comes to us. For example, when a user visits a website, we have no influence on the event. It just happens and we have to react to it. The challenge with online machine learning is to ensure that models trained offline on proactive datasets perform correctly in production on reactive data streams.

However, we train and derive the models online in the same order they would be used in production. This allows us to replicate a production scenario and evaluate models with greater fidelity than batch validation. This is what makes online machine learning so powerful. By reproducing data sets

in the correct order, we ensure that models will perform as expected in production.

VII. CONCLUSIONS

Today, new social engineering attacks continue to emerge, exposing end users to all kinds of losses, whether economic or social. Traditional security methods, such as operational mechanisms, do not solve the problem satisfactorily, especially for new attacks. Therefore, social engineering attack detection techniques based on machine learning address this problem in a more agile and effective way.

However, such approaches use batch processing for model training. This processing mode has the disadvantage of the model quickly becoming outdated, requiring high computational costs to retrain the model.

In this context, this paper presents a new technique that uses online training through algorithms on continuous data streams. The KNN, NB, LR and HATC algorithms were suitable to work in an online environment and combined with feature extraction schemes: BoW, TF-IDF and Word Embeddings. The analysis of the combinations of the algorithms with the feature extraction schemes showed that these models can be used efficiently and effectively in online training. A caveat is due to the HATC-spaCy model, where the results presented were not so satisfactory, in some metrics.

In future work, we will investigate how anomaly detection and clustering techniques can be applied to social engineering attacks using online training. In addition, as a technique aimed at large amounts of data, the deep learning approach to online training will be investigated. Finally, we will investigate how these techniques behave in a production environment by developing a security tool that can be run as a web application.

ACKNOWLEDGMENT

This work is co-financed by Component 5 - Capitalization and Business Innovation of core funding for Technology and Innovation Centres (CTI), integrated in the Resilience Dimension of the Recovery and Resilience Plan within the scope of the Recovery and Resilience Mechanism (MRR) of the European Union (EU), framed in the Next Generation EU, for the period 2021 – 2026.

REFERENCES

- [1] Pokrovskaya, N. Social engineering and digital technologies for the security of the social capital' development. In Proceedings of the International Conference of Quality Management, Transport and Information Security, Petersburg, Russia, 24–30 September 2017; pp. 16–19.
- [2] Steven C. H. Hoi, Doyen Sahoo, Jing Lu and Peilin Zhao. Online Learning: A Comprehensive Survey. *Neurocomputing*. Volume 459, 12 October 2021, Pages 249-289
- [3] Peng, T., Harris, I., & Sawa, Y. (2018). Detecting phishing attacks using natural language processing and machine learning. In 2018 IEEE 12th International Conference on Semantic Computing (ICSC) (pp 300–301).
- [4] P. Zambrano et al., "Technical mapping of the grooming anatomy using machine learning paradigms: An information security approach," *IEEE Access*, vol. 7, pp. 142129–142146, 2019.
- [5] Lansley, M.; Kapetanakis, S.; Polatidis, N. SEADer++ v2: Detecting Social Engineering Attacks using Natural Language Processing and Machine Learning. In Proceedings of the 2020 International Conference on INnovations in Intelligent SysTems and Applications (INISTA); IEEE: Novi Sad, Serbia, 2020; pp. 1–6.
- [6] Lansley, M., Polatidis, N., & Kapetanakis, S. (2019). Seader: A social engineering attack detection method based on natural language processing and artificial neural networks. In N. T. Nguyen, R. Chbeir, E. Exposito, P. Anioté, and B. Trawiński (Eds.), *Computational collective intelligence* (pp. 686–696). Springer International Publishing.
- [7] M. Lansley, F. Mouton, S. Kapetanakis, and N. Polatidis, "SEADer++: Social engineering attack detection in online environments using machine learning," *J. Inf. Telecommun.*, vol. 4, no. 3, pp. 346–362, 2020.
- [8] Lan, Y. Chat-Oriented Social Engineering Attack Detection Using Attention-based Bi-LSTM and CNN. In Proceedings of the 2021 2nd International Conference on Computing and Data Science (CDS), Stanford, CA, USA, 28–29 January 2021; pp. 483–487.
- [9] Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun. "Deep Residual Learning for Image Recognition". in Proceedings of the IEEE conference on computer vision and pattern recognition, 2016, pp. 770–778.
- [10] R.S.Michalski, J.G.Carbonell, T.M.Mitchell, *Machine learning: An artificial intelligence approach*, Springer Science & Business Media, 2013
- [11] Salahdine, F., & Kaabouch, N. (2019). Social engineering attacks: A survey. *Future Internet*, 11(4). <https://doi.org/10.3390/fi11040089>.
- [12] Foozy, C.F.M.; Ahmad, R.; Abdollah, M.F.; Yusof, R.; Mas'ud, M.Z. Generic taxonomy of social engineering attack and defense mechanism for handheld computer study. In Proceedings of the Malaysian Technical Universities International Conference on Engineering and Technology, Batu Pahat, Malaysia, 13–15 November 2011; pp. 1–6.
- [13] A. Lazarevic, L. Ertoz, V. Kumar, A. Ozgur, J. Srivastava, A Comparative Study of Anomaly Detection Schemes in Network Intrusion Detection, pp. 25–36.
- [14] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, A. Wesslén, *Experimentation in software engineering*, Springer Science & Business Media, 2012.
- [15] V. R. Basili, G. Caldiera, H. D. Rombach, The goal question metric approach, *Encyclopedia of software engineering* (1994) 528–532.
- [16] K. Weinberger, A. Dasgupta, J. Langford, A. Smola, and J. Attenberg, "Feature hashing for large scale multitask learning," in Proceedings of the 26th Annual International Conference on Machine Learning. ACM, 2009, pp. 1113–1120.
- [17] K. Jones, "A statistical interpretation of term specificity and its application in retrieval," *Journal of documentation*, vol. 28, no. 1, pp. 11–21, 1972.
- [18] Montiel, Jacob; Halford, Max; Mastelini, Saulo Martiello; Bolmier, Geoffrey; Sourty, Raphael; Vaysse, Robin; Zouitine, Adil; Gomes, Heitor Murilo; Read, Jesse; Abdessalem, Talel; Bifet, Albert. "River: machine learning for streaming data in Python", *Journal of Machine Learning Research* 22 (2021) 1-8
- [19] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, E. Duchesnay, "Scikit-learn: Machine learning in Python", *Journal of Machine Learning Research* 12 (2011) 2825–2830..
- [20] Bezuidenhout, Monique, Francois Mouton, and Hein S. Venter. "Social engineering attack detection model: Seadm." 2010 Information Security for South Africa. IEEE, 2010
- [21] Almeida, T.A., Hidalgo, J.M.G. and Yamakami, A., 2011, September. Contributions to the study of SMS spam filtering: new collection and results. In Proceedings of the 11th ACM symposium on Document engineering (pp. 259-262)